

Scenario 1: Sixty-eight thousand at Lumen Field

Slug: lumen-field-evac - **Repo:** github.com/joelcappgemini/titan-watch-lab - **Your role:** incident commander, full authority, next page

The kaiju, in thirty seconds

- **Gorathos.** Abyssal Colossus, 142 m, 88,000 t, 34.5 km/h. Walking ashore from Elliott Bay toward downtown through SoDo. Does not stop. Cannot be stopped. Anything it walks through is lost.
- **11 minutes** until it reaches the seawall. Straight-line estimate from Titan Watch, refreshed every 30 seconds. Firm on time, not on heading.
- **It is drawn toward dense noise and light.** The stadium is the loudest and brightest thing in the city. It does not pursue individuals.
- **Who sees it.** Titan Watch sees where it is, every 30 seconds. Stadium cameras see gates, police see streets, fans' phones see it first. Nobody sees which gate is safe. That gap is the scenario.

Sold out, 68,000 inside. Empty it into streets that face the kaiju, or hold them in a building never built to take a hit.

Outcome, same for every scenario

Save as much human life as possible, then prevent as much public property damage as possible, in that order. A software tool is expected to come out of this.

You are graded on, in this order: people killed or injured inside and outside the stadium; crush injuries at gates; damage to Lumen Field and SoDo; and whether every decision you made was a decision, recorded, with a name on it.

The coach interviews you against this, narrows it to this clock and this building, and restates it after synthesis. The original stays in the scope file beside the restatement.

Who reports to you

Nine specialists have already been interviewed. Their interviews are in `.copilot-tracking/dt/lumen-field-evac/`. Do not read them. The coach has, and will quiz you on what they said.

Reports to you	What they own	What they keep saying
Stadium operations director	Building, lights, PA, gate policy	Twenty-five minutes to clear, never timed
Gate and crowd-flow supervisor	Every gate, by radio	Hold or open lives on channel 2, no record, no name
Police incident lead	Streets, officers	Streets and gates are two lists, never reconciled
Fire battalion chief	Crush response	Two gate lines covered; hears the clock fourth-hand
PA and broadcast producer	One voice for 68,000	Cannot tell west from east
Sound Transit control	Trains, platform	2,500 southbound; nobody has called
Titan Watch analyst and liaison	The clock	Time is firm, heading bends toward the lights
A section steward, 300 level west	400 people, one stair	Silence means the nearest gate, and it faces the water

Already done, do not redo

Scope, stakeholder map, assumptions log, world rules, nine interviews, security scoping answers. All under `.copilot-tracking/dt/lumen-field-evac/`. The coach reads them.

Out of scope for everyone: engaging Gorathos, evacuating downtown, anything after it passes.

Your card: Incident commander, Lumen Field

Scenario 1, Lumen Field. Kaiju facts are on page 1 of this pack. Human facts are yours: invent and label.

Outcome: save as much human life as possible, then prevent as much public property damage as possible, in that order. The coach narrows it; that is fine.

Slug: lumen-field-evac-commander

Who you are

You are in charge. Stadium, police, fire, Sound Transit and the Titan Watch liaison all answer to you for the next 11 minutes. You can hold or release any gate, kill the lights, close any street, turn any train, send any crew anywhere. Nobody will argue. The only things you do not control are Gorathos and the clock.

Who you can call, and for what

- Operations director: building, lights, PA, time to clear.
- Gate supervisor: which gates are open or held.
- Police lead: which streets are clear.
- Fire chief: what they can cover.
- PA producer: what 68,000 people hear next.
- Sound Transit: how many the platform can take.
- Titan Watch liaison: the clock, the heading, how stale.
- Section steward: what one concourse is doing.

Each sees a piece. You see what they tell you, when they tell you. Your only screen is the Titan Watch command centre; everything else arrives by report, late.

Your scope

In scope, you decide and order:

- Hold or release, gate by gate, and when.
- Which sections go to which gate, which stay, what they are told.
- Lights, PA, streets, trains, crews, when to stop saving everyone and start choosing.

Out of scope:

- How a specialist carries out your order. They know their job.
- Where Gorathos is or goes. Titan Watch reports it; nobody controls it.

Anchors, fixed

- Every order you give takes minutes to reach the people who act on it, and every report reaches you late.
- No one has ever emptied this building against a clock.

Three phrases

"I do not know." - "Assume and label it." - "Ask the specialist, then tell me."

Next

Turn the page. The run sheet has every command and prompt for this scenario, in order.

How to proceed: Lumen Field

Order of work, with every prompt filled in for this scenario. Full detail is in the shared instructions.

Lab 1: Discovery

- 1 /clear, then /hve-core:dt-coach. It lists the projects it can resume. Pick yours, the one named on page 1. Then type: I am the incident commander. Start the interview. If no list appears, type resume project <name on page 1> first.
- 1 Answer five questions as the commander. Read your interview file once.
- 2 Same session, paste:

Project lumen-field-evac. Read the shared scope under .copilot-tracking/dt/lumen-field-evac/, every interview there, and my interview under .copilot-tracking/dt/lumen-field-evac-commander/. Restate the outcome in light of the interviews, keeping the original beside it. Summarise the problem as if hearing it for the first time. List what you still need before a PRD. Then draft the PRD with non-functional requirements, close gaps with labelled assumptions, and state what it rests on. Then a technology-neutral diagram as Mermaid. Then an opinionated Azure overlay with the minimum services, rejected alternatives recorded, and a latency budget. Write everything under .copilot-tracking/dt/lumen-field-evac/. Run without stopping. Label all interviews role-play.
- 1 PRD, diagram and overlay land under dt/lumen-field-evac/.

Lab 2: Security and Responsible AI

Write three predicted security findings on paper first. Two terminals.

- 1 Terminal 1: `/clear`, then `/hve-core:security-planner`. Paste:

Use the security planner to plan the production security for this solution. PRD and architecture are under `.copilot-tracking/dt/lumen-field-evac/`. Scoping answers are in `security-scoping-answers.md` in the same folder; treat every row as answered and labelled, do not ask them again. Threat-model the boundaries, map controls to OWASP, NIST and CIS, produce a backlog. Ask me only about severe or blocking issues. Run all phases without stopping. Prioritise by severity. Do not commit or push.

- 1 Terminal 2: `/clear`, then `/hve-core:rai-planner-agent`. Paste:

Run an RAI assessment on the single-save decision: the disaster-recovery AI in this app's Last Stand scene that recommends which one of two cities to save when only one can be saved. Today it outputs a single verdict with no confidence, no explanation and no audit trail, and a commander approves it under time pressure. Assess it against NIST AI RMF and treat the autonomous life-safety decision as the central risk. Use project name `single-save-decision`. Imagine the scoping answers, label them, run to completion unless blocking.

- 1 When the security planner finishes: "Plan summary by severity: counts per level, Critical items one line each, recommended next planner." Compare with your paper. Read the RAI go/no-go.

Lab 3: Research, Plan, Implement

- 1 Open your PRD under `dt/lumen-field-evac/`. Pick one functional requirement the app could carry. One screen, one rule, one feed.
- 2 `/clear`, then `/hve-core:rpi-agent`. Ask: "What do you need from me to create an RPI plan?" Then paste, filled in:

Goal: implement <requirement, one sentence> from the PRD at `.copilot-tracking/dt/lumen-field-evac/<prd file>`.
Scope: <one component or module>. Nothing else.
Acceptance criteria: <two or three testable lines>.
Constraints: no backend, no tracker, no cloud tenant; mock or defer and log the gap. No tracking paths in code or commits.
Validation: the existing test, lint and build scripts.
Context: the PRD and the Azure overlay in the same folder.
Mode: research, stop for my review; plan, stop for my approval.

- 1 Research stops: read it, fix the file if wrong, then `/hve-core:rpi-plan`.
- 2 Plan stops: amend one item in the plan file, tell the agent "plan amended in the file".
- 3 Homework: `/hve-core:rpi-implement`, then `/hve-core:rpi-review`.